

Лабораторная работа №7

Дискретно-событийное моделирование

Андрей Геннадьевич Софич

Содержание

1	Цель работы	4
2	Задание	5
3	Выполнение лабораторной работы	6
4	Выводы	46

Список иллюстраций

3.1	Инициализация проекта	6
3.2	Подключение необходимых пакетов	6
3.3	Создание файла для реализации M/M/c	7
3.4	Код в стиле jupyter notebook	7
3.5	Файл с графиками	8
3.6	Диаграмма обслуживания	8
3.7	Количество клиентов в системе	9
3.8	Время ожидания по клиентам	9
3.9	Код в стиле jupyter notebook	10
3.10	Первый прогон модели Росса	10
3.11	Код в стиле jupyter notebook	11
3.12	Результат анимации	11
3.13	Код в стиле jupyter notebook	12
3.14	Код в стиле jupyter notebook	12
3.15	Код в стиле jupyter notebook	13
3.16	Загрузка ремонтника	13
3.17	Код в стиле jupyter notebook	13
3.18	Добавление графика	14
3.19	График	14
3.20	Код в стиле jupyter notebook	15

1 Цель работы

Изучить основные модели теории массового обслуживания

2 Задание

Применить модели на задачах, выполнить несколько различных симуляций.

3 Выполнение лабораторной работы

Создаем рабочий каталог и инициализируем проект в julia (рис. 3.1).

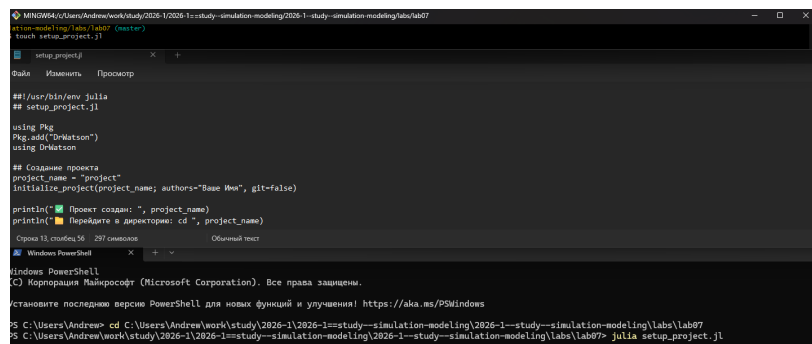


Рисунок 3.1: Инициализация проекта

Создаем файл, где прописываем пакеты, необходимые для выполнения работы и запускаем его (рис. 3.2).

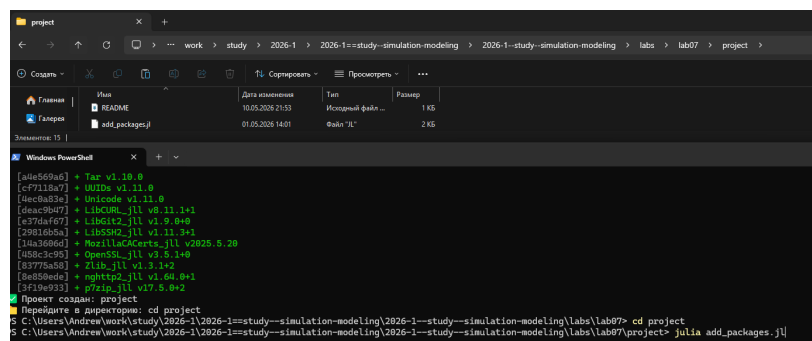


Рисунок 3.2: Подключение необходимых пакетов

Создаем файл в папке scripts, реализуем в нем модель M/M/c, которая будет симулировать и анализировать входящие потоки, обслуживания их интенсивности

и занятость приборов(каналов) на 10 клиентах(customers)и 2 серверах(каналах) (рис. 3.3).

```
MINGW64 ~/Users/Andrew/workstudy/2026-1-study-simulation-modeling/labs/lab7/project
#conda activate base2026
~/workstudy/2026-1-study-simulation-modeling/labs/lab7/project (master)
$python lab7.py 10 1000000
~/workstudy/2026-1-study-simulation-modeling/labs/lab7/project (master)

println("Customer %d entered service: ", now(env))
@yield timeout(env, rand(rng, d_s)) # server is busy
@yield unless(server) # customer exits service
println("Customer %d exited service: ", now(env))
end

# setup and run simulation
function setup_and_run()
    sim = Simulation{C} # initialize simulation environment
    server = Resource{cin, num_servers} # initialize servers
    arrival_time = 0.0
    for i = 1:num_customers # initialize customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dist)
    end
    run(sim) # run simulation
end
setup_and_run()

Copena Hl copena{11 12 as 1 389 cmeanos Ofsenal vect

Windows PowerShell X + v

(Pv1.12) pkg> C

Julia> exit()
PS C:\Users\Andrew\workstudy\2026-1-study-simulation-modeling\labs\lab7\project> julia scripts/file_01.jl
Customer 1 arrived: 0.1451845436852475
Customer 1 entered service: 0.1451845436852475
Customer 2 arrived: 0.5961331526335804
Customer 2 entered service: 0.5961331526335804
Customer 3 arrived: 1.5409684267811974
Customer 4 arrived: 1.6242796925312127
Customer 5 arrived: 1.691100878969648
Customer 1 exited service: 2.208985520126681
Customer 2 exited service: 2.208985520126681
Customer 6 arrived: 2.29898039524296317
Customer 3 exited service: 3.5822128399402174
Customer 4 exited service: 3.5822128399402174
Customer 7 arrived: 4.377380215158456
Customer 8 arrived: 5.16494277808802
Customer 2 exited service: 5.98672229377668
Customer 5 arrived service: 5.98672229377668
Customer 9 arrived: 7.0699994118638705
Customer 10 arrived: 7.828990226941469
Customer 6 exited service: 8.3315617262830
```

Рисунок 3.3: Создание файла для реализации М/М/с

Преобразовываем код в произвольные стили и открываем один из них, чтобы убедиться в правильности компиляции (рис. 3.4).

```

C:\Users\Andrew\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project\wardown\file_01\file_01.qmd
In [ ]: # Run C Users\Andrew\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project\wardown\file_01.jl
File writing result to: C:\Users\Andrew\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project\notebooks\file_01\file_01.ipynb
~/Notebook: C:\Users\Andrew\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project\notebooks\file_01\file_01.ipynb

Format! like gnu/gnu coqasm.
PS C:\Users\Andrew\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project>

C:\Users\Andrew>

```

Рисунок 3.4: Код в стиле `jupyter notebook`

Создаем файл, в котором добавляем графики: диаграмма обслуживания, кол-во клиентов в системе, время ожидания по клиентам (рис. 3.5).

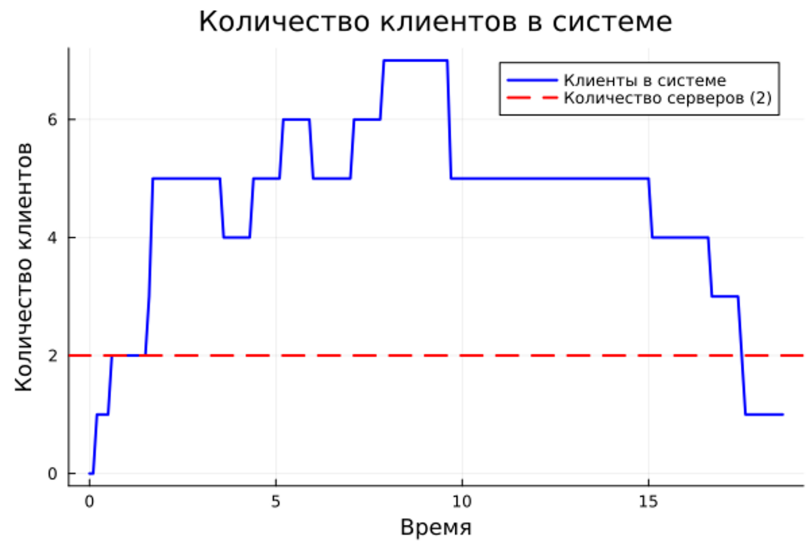


Рисунок 3.7: Количество клиентов в системе

Третий график: время ожидания по клиентам - мы видим, как скапливаются клиенты, образуются очереди и сколько customers приходится ждать пока освободится сервер (рис. 3.8).

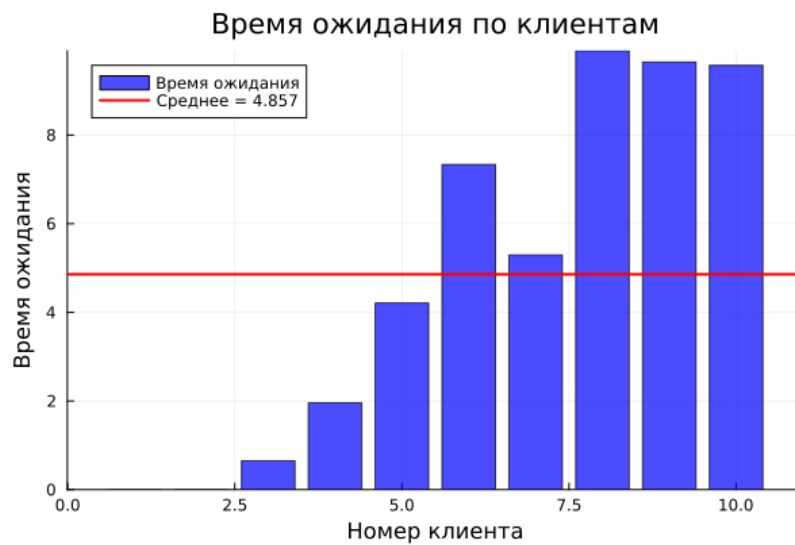


Рисунок 3.8: Время ожидания по клиентам

Преобразовываем код в произвольные стили и открываем один из них, чтобы убедиться в правильности компиляции (рис. 3.9).

```
Q: quart: C:\Users\Andrey\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project\notebooks\file_graphs\file_graphs.pyd
Info: generating notebook from C:\Users\Andrey\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project\notebooks\file_graphs\file_graphs.jl
Info: writing result to C:\Users\Andrey\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project\notebooks\file_graphs\file_graphs.ipynb
* Notebook: C:\Users\Andrey\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project\notebooks\file_graphs\file_graphs.ipynb

[Output] Run file_graphs.ipynb
PS C:\Users\Andrey\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project>

file_graphs.pyd
C:\Users\Andrey\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project> file_graphs.pyd
Generate: 1 Code 1 Markdown 1 Run All 1 Output 1 Clear All Output 1 Outline
Customer 1 arrived: 1.4811886768680648
Customer 1 exited service: 2.3086521021210481
Customer 1 reserved service: 2.3086521021210481
Customer 2 arrived: 2.3086521021210481
Customer 2 exited service: 3.52223899442124
Customer 3 arrived: 4.1779261158465
Customer 4 arrived: 5.546427576682
Customer 4 exited service: 5.98672292577648
Customer 5 arrived service: 5.98672292577648
Customer 6 arrived: 7.489994181367876
Customer 6 arrived: 7.489994181367876
Customer 6 exited service: 9.4345964788254
Customer 6 reserved service: 9.4345964788254
Customer 6 exited service: 17.4883114476128
Customer 10 arrived service: 17.4883114476128
Customer 9 arrived: 17.58865112115993
Customer 9 reserved service: 17.58865112115993
Customer 10 arrived: 18.58865112115993
```

Рисунок 3.9: Код в стиле jupyter notebook

Переходим в модель Росса, для начала реализуем обычные симуляции с параметрами (рабочие машины, запасные, время починки, время поломки и тд.) (рис. 3.10).

```
MINOWA4\Users\Andrey\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project
C:\Users\Andrey\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project (master)
touch scripts/ross_1.jl
C:\Users\Andrey\work\study\2026-1\2026-1-study-simulation-modeling\2026-1-study-simulation-modeling\labs\lab07\project (master)

file_project | add_package.jl | file_01.jl | file_graph.jl | ross_1.jl |
Файл | Имя | Просмотр
spares = Store{Process}()
for i = 1:n
    proc = @process machine(env, repair_facility, spares)
    @yield interrupt(proc)
end
for i = 1:5
    proc = @process machine(env, repair_facility, spares)
    @yield put!(spares, proc)
end
end
function sim_repair()
    sim = Simulation()
    repair_facility = Resource(sim)
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares)
    res = run(sim)
    stop_time = now(sim)
end
Строка 68, столбец 4 1/738 символов Обычный текст

Windows PowerShell
At time 12715.718224958666: No more spares!
At time 37335.53567595887: No more spares!
At time 38888.52467877361: No more spares!
At time 1681.2520911770555: No more spares!
At time 824.1948788485848: No more spares!
Average crash time: 16664.247589204883
```

Рисунок 3.10: Первый прогон модели Росса

Преобразовываем код в произвольные стили и открываем один из них, чтобы убедиться в правильности компиляции (рис. 3.11).

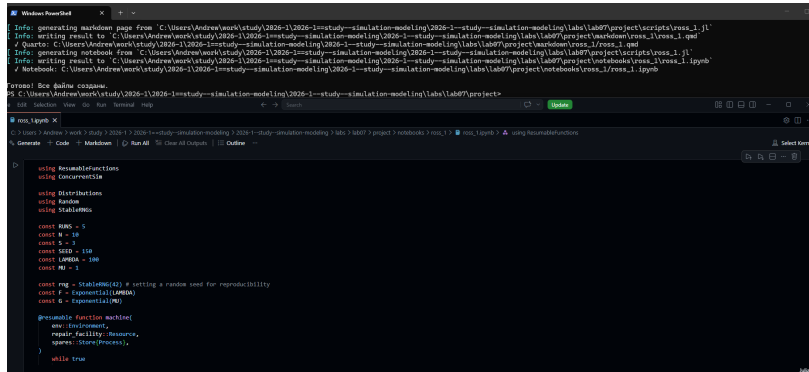


Рисунок 3.11: Код в стиле jupyter notebook

Преобразовываем код, добавляем ремонтников, я решил взять 5, как мы видим, время работы системы сильно увеличилось (рис. 3.12).

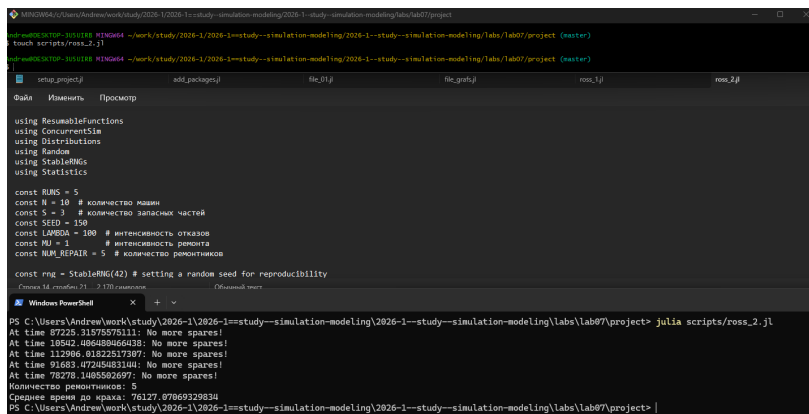


Рисунок 3.12: Результат анимации

Преобразовываем код в произвольные стили и открываем один из них, чтобы убедиться в правильности компиляции (рис. 3.13).

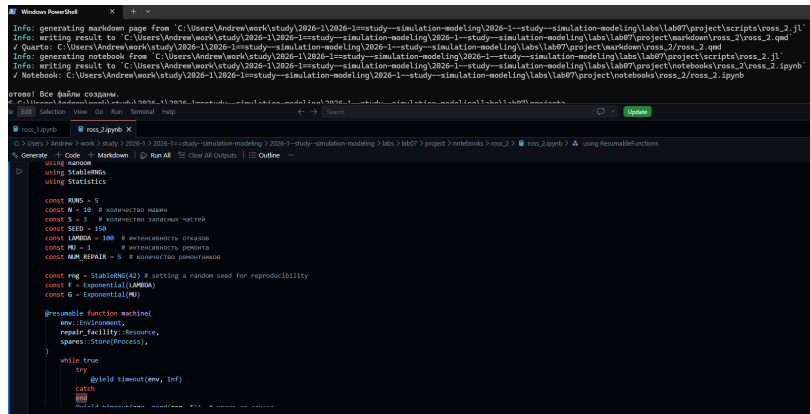


Рисунок 3.13: Код в стиле jupyter notebook

Теперь проводим симуляцию с разным количеством работающих машин($N=30$), оставляем резерв таким же, как мы видим, система довольно быстро вышла из строя, что не удивительно (рис. 3.14).

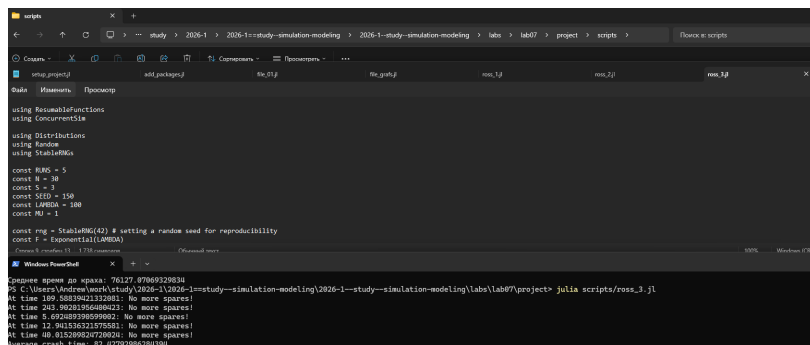


Рисунок 3.14: Код в стиле jupyter notebook

Преобразовываем код в произвольные стили и открываем один из них, чтобы убедиться в правильности компиляции (рис. 3.15).

Создаем график изменения числа исправных машин во времени (рис. 3.18).

```

MINKW04/Users/andrew/workstudy/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/lab07/project
~/workstudy/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/lab07/project (master)
touch script/rsa.5.11
~/workstudy/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/lab07/project (master)
vim script/rsa.5.11
add_plot.py
end
line([total_machines], linestyle='dash', label='Всего машин', color='green')
line([N], linestyle='dot', label='Рабочих машин (N)', color='orange')
savefig(p3, 'plots/combined_state.png')
print("\n plots/combined_state.png")

# Дополнительная статистика
print("\n==== СТАТИСТИКА ПОСЛЕДНЕЙ СИМУЛЯЦИИ ===")
print("Время остановки: ${round(all_stop_times[end], digits=3)}")
print("Средняя длина очереди: ${round(mean(queue_lengths), digits=3)}")
print("Макс. длина очереди: ${round(maximum(queue_lengths), digits=3)}")
print("Минимум исправных машин: ${round(minimum(working_machines), digits=3)}")
print("Среднее исправных машин: ${round(mean(working_machines), digits=2)}")
print("\nВсе графики сохранены в папку 'plots'")

Строка 103, столбец 31 8 407 символов Обычный текст

Windows PowerShell
At time 1681.2524911974856: No more spares!
--- Симуляция 5 ---
At time 524.1848780485848: No more spares!
Average crash time: 16664.247588264883

=== ПОСТРОЕНИЕ ГРАФИКОВ ===
✓ plots/working_machines_time.png
✓ plots/queue_length_time.png
✓ plots/combined_state.png

=== СТАТИСТИКА ПОСЛЕДНЕЙ СИМУЛЯЦИИ ===
Время остановки: 524.185
Средняя длина очереди: 9.618
Макс. длина очереди: 2.0
Минимум исправных машин: 9.0
Среднее исправных машин: 12.86
Все графики сохранены в папку 'plots'

```

Рисунок 3.18: Добавление графика

Как мы видим, очередь на ремонт большая и рано или поздно система встает (машина сломалась но её нельзя заменить резервной) (рис. 3.19).

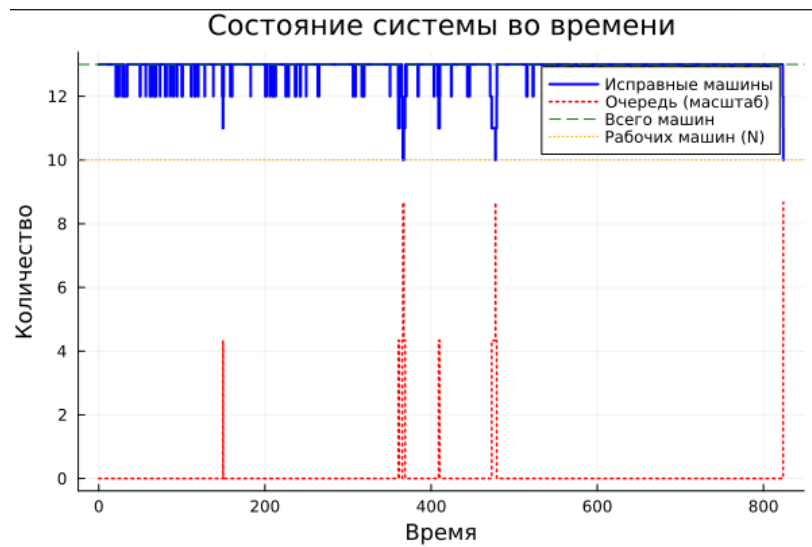


Рисунок 3.19: График

Преобразовываем код в произвольные стили и открываем один из них, чтобы убедиться в правильности компиляции (рис. 3.20).

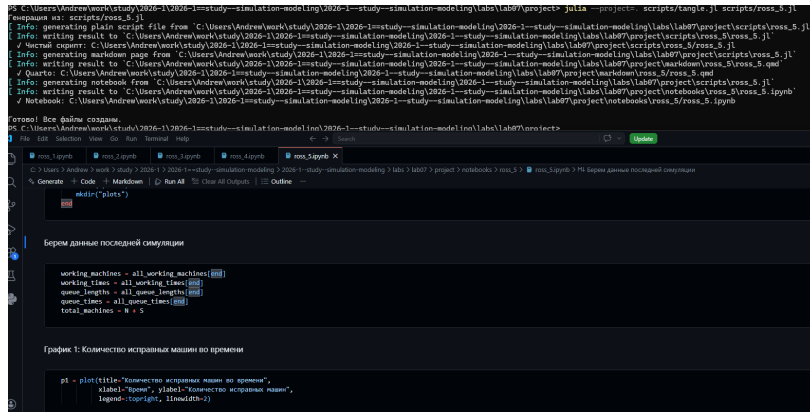


Рисунок 3.20: Код в стиле jupyter notebook

Для первого прогона модели М/М/с

```
using StableRNGs
using Distributions
using ConcurrentSim
using ResumableFunctions

#set simulation parameters
rng = StableRNG(123)
num_customers = 10 # total number of customers generated
```

set queue parameters

```
num_servers = 2 # number of servers
mu = 1.0 / 2 # service rate
lam = 0.9 # arrival rate
arrival_dist = Exponential(1 / lam) # interarrival time distribution
service_dist = Exponential(1 / mu) # service time distribution
```

define customer behavior

```

@resumable function customer(
    env::Environment,
    server::Resource,
    id::Integer,
    t_a::Float64,
    d_s::Distribution,
)
    @yield timeout(env, t_a) # customer arrives
    println("Customer $id arrived: ", now(env))
    @yield request(server) # customer starts service
    println("Customer $id entered service: ", now(env))
    @yield timeout(env, rand(rng, d_s)) # server is busy
    @yield unlock(server) # customer exits service
    println("Customer $id exited service: ", now(env))
end

```

setup and run simulation

```

function setup_and_run()
    sim = Simulation() # initialize simulation environment
    server = Resource(sim, num_servers) # initialize servers
    arrival_time = 0.0
    for i = 1:num_customers # initialize customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dist)
    end
    run(sim) # run simulation
end

setup_and_run()

```


Построения графиков М/М/с

```
using StableRNGs
using Distributions
using ConcurrentSim
using ResumableFunctions
using Plots
using Statistics

#set simulation parameters
rng = StableRNG(123)
num_customers = 10 # total number of customers generated
```

set queue parameters

```
num_servers = 2 # number of servers
mu = 1.0 / 2 # service rate
lam = 0.9 # arrival rate
arrival_dist = Exponential(1 / lam) # interarrival time distribution
service_dist = Exponential(1 / mu) # service time distribution
```

Массивы для сбора данных

```
arrival_times = Vector{Float64}(undef, num_customers)
service_start_times = Vector{Float64}(undef, num_customers)
service_end_times = Vector{Float64}(undef, num_customers)
```

define customer behavior

```

@resumable function customer(
    env::Environment,
    server::Resource,
    id::Integer,
    t_a::Float64,
    d_s::Distribution,
)

    @yield timeout(env, t_a) # customer arrives
    arrival_times[id] = now(env)
    println("Customer $id arrived: ", now(env))
    @yield request(server) # customer starts service
    service_start_times[id] = now(env)
    println("Customer $id entered service: ", now(env))
    service_time = rand(rng, d_s)
    @yield timeout(env, service_time) # server is busy
    service_end_times[id] = now(env)
    println("Customer $id exited service: ", now(env))
    @yield unlock(server) # customer exits service
end

```

setup and run simulation

```

function setup_and_run()
    sim = Simulation() # initialize simulation environment
    server = Resource(sim, num_servers) # initialize servers
    arrival_time = 0.0
    for i = 1:num_customers # initialize customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dis

```

```

end
run(sim) # run simulation

```

Создание графиков

```

waiting_times = service_start_times .- arrival_times

```

1. Диаграмма Ганта

```

p1 = plot(title="Гант-диаграмма",
           xlabel="Время", ylabel="Клиенты", legend=:topright, size=(800, 400))
for i in 1:num_customers
    plot!([arrival_times[i], service_start_times[i], service_end_times[i]],
          [i, i, i], marker=:circle, linewidth=2, label="Клиент $i")
end
savefig(p1, "plots/gantt_chart.png")
println("Гант-диаграмма сохранена в plots/gantt_chart.png")

```

2. Гистограмма времени ожидания

```

p2 = bar(1:length(waiting_times), waiting_times,
         title="Гистограмма времени ожидания",
         xlabel="Время ожидания", ylabel="Частота",
         label="Частота", color=:blue, alpha=0.7)
hline!([mean(waiting_times)], label="Среднее время ожидания = $(round(mean(waiting_times), 2))",
        color=:red, linewidth=2)
savefig(p2, "plots/waiting_time_histogram.png")
println("Гистограмма времени ожидания сохранена в plots/waiting_time_histogram.png")

```

3. Динамика количества клиентов

```

time_points = 0:0.1:maximum(service_end_times)
n_clients = zeros(length(time_points))

for j in 1:length(time_points)
    t = time_points[j]
    count = 0
    for i in 1:num_customers
        if arrival_times[i] <= t && service_end_times[i] > t
            count += 1
        end
    end
    n_clients[j] = count
end

p3 = plot(time_points, n_clients,
          title="Средняя нагрузка системы в зависимости от времени",
          xlabel="Время", ylabel="Средняя нагрузка системы",
          label="Средняя нагрузка системы в зависимости от времени", linewidth=2, color=:blue)
hline!([num_servers], linestyle=:dash, linewidth=2,
       label="Максимальная нагрузка системы ($num_servers)", color=:red)
savefig(p3, "plots/system_load.png")
println("Средняя нагрузка системы в зависимости от времени")

```

Краткая статистика

```

println("\n=== Средние значения ===")
println("Среднее время ожидания: $(round(mean(waiting_times), digits=2))")
println("Максимальное время ожидания: $(round(maximum(waiting_times), digits=2))")
println("Максимальное время обслуживания: $(round(maximum(service_end_times), digits=2))")

```

```
end
```

```
setup_and_run()
```

Первый прогон модели Росса

```
using ResumableFunctions
```

```
using ConcurrentSim
```

```
using Distributions
```

```
using Random
```

```
using StableRNGs
```

```
const RUNS = 5
```

```
const N = 10
```

```
const S = 3
```

```
const SEED = 150
```

```
const LAMBDA = 100
```

```
const MU = 1
```

```
const rng = StableRNG(42) # setting a random seed for reproducibili
```

```
const F = Exponential(LAMBDA)
```

```
const G = Exponential(MU)
```

```
@resumable function machine(  
    env::Environment,  
    repair_facility::Resource,  
    spares::Store{Process},  
)
```

```

while true
  try
    @yield timeout(env, Inf)
  catch
  end
  @yield timeout(env, rand(rng, F))
  get_spare = take!(spares)
  @yield get_spare | timeout(env)
  if state(get_spare) != ConcurrentSim.idle
    @yield interrupt(value(get_spare))
  else
    throw(StopSimulation("No more spares!"))
  end
  @yield request(repair_facility)
  @yield timeout(env, rand(rng, G))
  @yield unlock(repair_facility)
  @yield put!(spares, active_process(env))
end
end

@resumable function start_sim(
  env::Environment,
  repair_facility::Resource,
  spares::Store{Process},
)
  for i = 1:N
    proc = @process machine(env, repair_facility, spares)
    @yield interrupt(proc)
  end
end

```

```

    end
    for i = 1:S
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end

function sim_repair()
    sim = Simulation()
    repair_facility = Resource(sim)
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares)
    msg = run(sim)
    stop_time = now(sim)
    println("At time $stop_time: $msg")
    stop_time
end

results = Float64[]
for i = 1:RUNS
    push!(results, sim_repair())
end
println("Average crash time: ", sum(results)/RUNS)

```

Прогон модели Росса для 5 ремонтников

```

using ResumableFunctions
using ConcurrentSim
using Distributions

```

```

using Random
using StableRNGs
using Statistics

const RUNS = 5
const N = 10 # XXXXXXXXXXXX XXXX
const S = 3 # XXXXXXXXXXXX XXXXXXXXXXXX XXXXXXX
const SEED = 150
const LAMBDA = 100 # XXXXXXXXXXXXXXXX XXXXXXX
const MU = 1 # XXXXXXXXXXXXXXXX XXXXXXX
const NUM_REPAIR = 5 # XXXXXXXXXXXX XXXXXXXXXXXXXXX

const rng = StableRNG(42) # setting a random seed for reproducibili
const F = Exponential(LAMBDA)
const G = Exponential(MU)

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    while true
        try
            @yield timeout(env, Inf)
        catch
            end
            @yield timeout(env, rand(rng, F)) # XXXXX XX XXXXXXX
            get_spare = take!(spares)

```



```
@yield get_spare | timeout(env)
if state(get_spare) != ConcurrentSim.idle
    @yield interrupt(value(get_spare))
else
    throw(StopSimulation("No more spares!"))
end

@yield request(repair_facility) # [X][X][X][X] [X][X][X][X][X][X][X][X]
@yield timeout(env, rand(rng, G)) # [X][X][X][X] [X][X][X][X][X][X]
@yield unlock(repair_facility) # [X][X][X][X][X][X][X][X][X][X][X][X][X][X][X][X]
@yield put!(spares, active_process(env)) # [X][X][X][X][X][X] [ ] [X][X][X][X]

end

end

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
```

Запускаем N машин

```
for i = 1:N
    proc = @process machine(env, repair_facility, spares)
    @yield interrupt(proc)
end
```

Добавляем S запасных частей

```

    for i = 1:S
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end

function sim_repair()
    sim = Simulation()
    repair_facility = Resource(sim, NUM_REPAIR) # NUM_REPAIR
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares)
    msg = run(sim)
    stop_time = now(sim)
    println("At time $stop_time: $msg")
    return stop_time
end

results = Float64[]
for i = 1:RUNS
    push!(results, sim_repair())
end

println("XXXXXXXXXX XXXXXXXXXXXXX: $NUM_REPAIR")
println("XXXXXXXX XXXXX XX XXXXX: ", sum(results)/RUNS)

```

Прогон для 30 машин

```

using ResumableFunctions
using ConcurrentSim

using Distributions
using Random
using StableRNGs

const RUNS = 5
const N = 30
const S = 3
const SEED = 150
const LAMBDA = 100
const MU = 1

const rng = StableRNG(42) # setting a random seed for reproducibili
const F = Exponential(LAMBDA)
const G = Exponential(MU)

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    while true
        try
            @yield timeout(env, Inf)
        catch
        end
    end
end

```

```

    @yield timeout(env, rand(rng, F))
    get_spare = take!(spares)
    @yield get_spare | timeout(env)
    if state(get_spare) != ConcurrentSim.idle
        @yield interrupt(value(get_spare))
    else
        throw(StopSimulation("No more spares!"))
    end
    @yield request(repair_facility)
    @yield timeout(env, rand(rng, G))
    @yield unlock(repair_facility)
    @yield put!(spares, active_process(env))
end
end

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)
    for i = 1:N
        proc = @process machine(env, repair_facility, spares)
        @yield interrupt(proc)
    end
    for i = 1:S
        proc = @process machine(env, repair_facility, spares)
        @yield put!(spares, proc)
    end
end

```

```

end

function sim_repair()
    sim = Simulation()
    repair_facility = Resource(sim)
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares)
    msg = run(sim)
    stop_time = now(sim)
    println("At time $stop_time: $msg")
    stop_time
end

results = Float64[]
for i = 1:RUNS
    push!(results, sim_repair())
end
println("Average crash time: ", sum(results)/RUNS)

```

Прогон для загрузки ремонтника

```

using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs
using Statistics

const RUNS = 5

```



```

    catch
    end
    @yield timeout(env, rand(rng, F))
    get_spare = take!(spares)
    @yield get_spare | timeout(env)
    if state(get_spare) != ConcurrentSim.idle
        @yield interrupt(value(get_spare))
    else
        throw(StopSimulation("No more spares!"))
    end
end

```

Запрос ремонтника и запись длины очереди

```

current_queue = length(repair_facility.queue)
push!(queue_times, now(env))
push!(queue_lengths, current_queue)

@yield request(repair_facility)

```

Ремонт

```

    repair_time = rand(rng, G)
    @yield timeout(env, repair_time)

    @yield unlock(repair_facility)
    @yield put!(spares, active_process(env))
end
end

@resumable function monitor_system(

```

```

env::Environment,
repair_facility::Resource
)

while true
    @yield timeout(env, 0.1) # XXXXXXXXXXX XXXXXX 0.1 XXXXXXXX
    current_queue = length(repair_facility.queue)
    push!(queue_times, now(env))
    push!(queue_lengths, current_queue)
end
end

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
)

```

Запускаем мониторинг

```

@process monitor_system(env, repair_facility)

for i = 1:N
    proc = @process machine(env, repair_facility, spares)
    @yield interrupt(proc)
end
for i = 1:S
    proc = @process machine(env, repair_facility, spares)
    @yield put!(spares, proc)
end

```



```
end
```

```
function sim_repair()
```

```
    global queue_lengths, queue_times, repair_busy_time, last_state
```

Сбрасываем данные

```
    queue_lengths = Float64[]
```

```
    queue_times = Float64[]
```

```
    repair_busy_time = 0.0
```

```
    last_state_change = 0.0
```

```
    sim = Simulation()
```

```
    repair_facility = Resource(sim, NUM_REPAIR)
```

```
    spares = Store{Process}(sim)
```

```
    @process start_sim(sim, repair_facility, spares)
```

```
    msg = run(sim)
```

```
    stop_time = now(sim)
```

Расчет метрик

```
    avg_queue_length = length(queue_lengths) > 0 ? mean(queue_lengths)
```

```
    max_queue_length = length(queue_lengths) > 0 ? maximum(queue_lengths)
```

Расчет загрузки ремонтника через статистику ресурса ConcurrentSim не дает прямой доступ к времени занятости, оцениваем косвенно

```
    utilization = (1 - (length(repair_facility.queue) / (NUM_REPAIR
```

Альтернативный расчет загрузки через количество обращений

```

total_services = length(repair_facility.server) # 1000000

println("\n=== 1000000 Servers ===")
println("Time to repair: $(round(stop_time, digits=3))")
println("Message: $msg")
println("\n=== 1000000 Servers ===")
println("Average queue length: $(round(avg_queue_length, digits=3))")
println("Maximum queue length: $max_queue_length")
println("Number of servers: $(length(queue_lengths))")
println("\n=== 1000000 Servers ===")
println("Number of repairs: $NUM_REPAIR")
println("Utilization: $(round(utilization, digits=2))%")

return stop_time, avg_queue_length, max_queue_length, utilization
end

```

Запуск нескольких симуляций

```

results_time = Float64[]
results_queue = Float64[]
results_maxqueue = Float64[]
results_util = Float64[]

println("Time to repair: $(RUNS) 1000000 Servers...\n")
println("="^60)

for i = 1:RUNS
    println("\n--- 1000000 Servers ---")
    stop_time, avg_q, max_q, util = sim_repair()

```

```

push!(results_time, stop_time)
push!(results_queue, avg_q)
push!(results_maxqueue, max_q)
push!(results_util, util)
end

```

Итоговая статистика

```

println("\n" * "="^60)
println("XXXXXXXXXX XXXXXXXXXXXX XX $(RUNS) XXXXXXXX (1 XXXXXXXXXXXX)")
println("="^60)

println("\n=== XXXXX XX XXXXX ===")
println("XXXXXXXXXX: $(round(mean(results_time), digits=3))")
println("XX. XXXXXXXXXXXX: $(round(std(results_time), digits=3))")
println("XXXXXXXXXX: $(round(minimum(results_time), digits=3))")
println("XXXXXXXXXX: $(round(maximum(results_time), digits=3))")

println("\n=== XXXXXXXX XXXXX XXXXXXXX ===")
println("XXXXXXXXXX: $(round(mean(results_queue), digits=3))")
println("XX. XXXXXXXXXXXX: $(round(std(results_queue), digits=3))")
println("XXXXXXXXXX: $(round(minimum(results_queue), digits=3))")
println("XXXXXXXXXX: $(round(maximum(results_queue), digits=3))")

println("\n=== XXXXXXXXXXXXXXXX XXXXX XXXXXXXX ===")
println("XXXXXXXXXX: $(round(mean(results_maxqueue), digits=3))")
println("XX. XXXXXXXXXXXX: $(round(std(results_maxqueue), digits=3))")

println("\n=== XXXXXXXX XXXXXXXXXXXX ===")

```

```
println("Среднее: $(round(mean(results_util), digits=2))%")
println("Станд. откл.: $(round(std(results_util), digits=2))%")
println("Минимум: $(round(minimum(results_util), digits=2))%")
println("Максимум: $(round(maximum(results_util), digits=2))%")

println("\n" * "="^60)
println("Результаты:")
println("1. Среднее время = $(round(mean(results_time), digits=2))")
println("2. Среднее количество машин в очереди = $(round(mean(results_queue), digits=3))")
println("3. Среднее количество машин в эксплуатации = $(round(mean(results_util), digits=2))")
```

Добавление графика изменения числа машин во времени

```
using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs
using Plots
using Statistics

const RUNS = 5
const N = 10
const S = 3
const SEED = 150
const LAMBDA = 100
const MU = 1

const rng = StableRNG(42) # setting a random seed for reproducibili
```

```
const F = Exponential(LAMBDA)
const G = Exponential(MU)
```

Глобальные переменные для сбора данных

```
all_working_machines = Vector{Vector{Float64}}()
all_working_times = Vector{Vector{Float64}}()
all_queue_lengths = Vector{Vector{Float64}}()
all_queue_times = Vector{Vector{Float64}}()
```

Счетчик машин в очереди

```
queue_counter = 0

@resumable function machine(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
    working_machines::Vector{Float64},
    working_times::Vector{Float64},
    queue_lengths::Vector{Float64},
    queue_times::Vector{Float64},
    total_machines::Int,
)

    global queue_counter

    while true
        try
            @yield timeout(env, Inf)
```

```
catch
end
```

Машина работает

```
@yield timeout(env, rand(rng, F))
```

Поломка - уменьшаем исправные машины

```
push!(working_times, now(env))
push!(working_machines, working_machines[end] - 1)

get_spare = take!(spares)
@yield get_spare | timeout(env)
if state(get_spare) != ConcurrentSim.idle
    @yield interrupt(value(get_spare))
else
    throw(StopSimulation("No more spares!"))
end
```

Запись очереди перед запросом ремонтника

```
push!(queue_times, now(env))
push!(queue_lengths, queue_counter)

queue_counter += 1 # XXXXXX X XXXXXXXX
@yield request(repair_facility)
queue_counter -= 1 # XXXXXXXX XX XXXXXXXX

@yield timeout(env, rand(rng, G))
@yield unlock(repair_facility)
```

Ремонт завершен - увеличиваем исправные машины

```
    push!(working_times, now(env))
    push!(working_machines, working_machines[end] + 1)

    @yield put!(spares, active_process(env))
end
end

@resumable function monitor_system(
    env::Environment,
    repair_facility::Resource,
    working_machines::Vector{Float64},
    working_times::Vector{Float64},
    queue_lengths::Vector{Float64},
    queue_times::Vector{Float64},
    total_machines::Int,
)
    global queue_counter

    while true
        @yield timeout(env, 0.1)
```

Запись текущей длины очереди

```
    push!(queue_times, now(env))
    push!(queue_lengths, queue_counter)
```

Запись исправных машин

```

        push!(working_times, now(env))
        push!(working_machines, working_machines[end])
    end
end

@resumable function start_sim(
    env::Environment,
    repair_facility::Resource,
    spares::Store{Process},
    working_machines::Vector{Float64},
    working_times::Vector{Float64},
    queue_lengths::Vector{Float64},
    queue_times::Vector{Float64},
    total_machines::Int,
)

```

Запускаем мониторинг

```

@process monitor_system(env, repair_facility, working_machines,
                        queue_lengths, queue_times, total_machines)

for i = 1:N
    proc = @process machine(env, repair_facility, spares, working_machines,
                           working_times, queue_lengths, queue_times)
    @yield interrupt(proc)
end

for i = 1:S
    proc = @process machine(env, repair_facility, spares, working_machines,
                           working_times, queue_lengths, queue_times)

```



```

        @yield put!(spares, proc)
    end
end

function sim_repair()
    global queue_counter
    queue_counter = 0

    total_machines = N + S
    working_machines = [Float64(total_machines)]
    working_times = [0.0]
    queue_lengths = Float64[]
    queue_times = Float64[]

    sim = Simulation()
    repair_facility = Resource(sim) # 1 XXXXXXXXXXXX
    spares = Store{Process}(sim)
    @process start_sim(sim, repair_facility, spares, working_machines,
                      queue_lengths, queue_times, total_machines)

    msg = run(sim)
    stop_time = now(sim)
    println("At time $stop_time: $msg")
end

```

Сохраняем данные для графиков

```

push!(all_working_machines, copy(working_machines))
push!(all_working_times, copy(working_times))
push!(all_queue_lengths, copy(queue_lengths))
push!(all_queue_times, copy(queue_times))

```

```

    return stop_time, working_machines, working_times, queue_length
end

```

Запуск симуляций и сбор данных

```

results = Float64[]
all_stop_times = Float64[]

println("##### $(RUNS) #####...\n")

for i = 1:RUNS
    println("--- ##### $i ---")
    stop_time, _, _, _, _ = sim_repair()
    push!(results, stop_time)
    push!(all_stop_times, stop_time)
end

println("\nAverage crash time: ", sum(results)/RUNS)

```

Построение графиков по последней симуляции

```

println("\n=== ##### ===")

```

Создаем папку для графиков

```

if !isdir("plots")
    mkdir("plots")
end

```

Берем данные последней симуляции

```

working_machines = all_working_machines[end]
working_times = all_working_times[end]
queue_lengths = all_queue_lengths[end]
queue_times = all_queue_times[end]
total_machines = N + S

```

График 1: Количество исправных машин во времени

```

p1 = plot(title="График 1: Количество исправных машин во времени",
          xlabel="Время", ylabel="Количество исправных машин",
          legend=:topright, linewidth=2)

```

Подготовка данных для ступенчатого графика

```

plot_times = []
plot_values = []
for i in 1:length(working_times)-1
    push!(plot_times, working_times[i])
    push!(plot_values, working_machines[i])
    push!(plot_times, working_times[i+1])
    push!(plot_values, working_machines[i])
end

plot!(plot_times, plot_values, label="Количество исправных машин", color=:blue,
      hline!([total_machines], linestyle=:dash, label="Общее количество машин ($total_machines)",
      hline!([N], linestyle=:dot, label="Количество машин (N=$N)", color=:orange)
savefig(p1, "plots/working_machines_time.png")
println("Файл plots/working_machines_time.png")

```

График 2: Длина очереди на ремонт

```
p2 = plot(title="Среднее время ожидания в очереди",
          xlabel="Время", ylabel="Среднее время ожидания в очереди",
          legend=:topright, linewidth=2)
```

Подготовка данных для ступенчатого графика очереди

```
if length(queue_times) > 0
    queue_times_plot = []
    queue_vals_plot = []
    for i in 1:length(queue_times)-1
        push!(queue_times_plot, queue_times[i])
        push!(queue_vals_plot, queue_lengths[i])
        push!(queue_times_plot, queue_times[i+1])
        push!(queue_vals_plot, queue_lengths[i])
    end

    plot!(queue_times_plot, queue_vals_plot, label="Среднее время ожидания в очереди", co
    if length(queue_lengths) > 0
        hline!([mean(queue_lengths)], linestyle=:dash,
               label="Среднее время ожидания в очереди = $(round(mean(queue_lengths), digits=2)
    end
end

savefig(p2, "plots/queue_length_time.png")
println(" plots/queue_length_time.png")
```

График 3: Совмещенный график

```
p3 = plot(title="Среднее время ожидания в очереди",
          xlabel="Время", ylabel="Среднее время ожидания в очереди",
          legend=:topright)
```

Исправные машины

```
plot!(plot_times, plot_values, label="XXXXXXXXXX XXXXXXX", color=:blue, s
```

Очередь (масштабируем для наглядности)

```

if length(queue_lengths) > 0 && maximum(queue_lengths) > 0
    scale_factor = total_machines / (maximum(queue_lengths) + 1)
    queue_scaled = queue_vals_plot .* scale_factor
    plot!(queue_times_plot, queue_scaled, label="XXXXXXXX (XXXXXXXX)", c
end

hline!([total_machines], linestyle=:dash, label="XXXXXX XXXXXX", color=
hline!([N], linestyle=:dot, label="XXXXXXXX XXXXXX (N)", color=:orange)
savefig(p3, "plots/combined_state.png")
println("✶ plots/combined_state.png")

```

Дополнительная статистика

```
println("\n=== 队列长度统计 ===")
println("队列长度: $(round(all_stop_times[end], digits=3))")
println("平均队列长度: $(round(mean(queue_lengths), digits=3))")
println("最大队列长度: $(round(maximum(queue_lengths), digits=0))")
println("最小工作机器数: $(round(minimum(working_machines), digits=3))")
println("平均工作机器数: $(round(mean(working_machines), digits=3))")

println("\n队列长度统计图 已生成 'plots'")
```

4 Выводы

В данной работе мы изучили и применили дискретно-событийное моделирование.